Fine-tuning Whisper STT on Trump Speeches

John Foley

Table of Contents

Whisper STT 2
Task / Dataset Selection 2
Low Rank Adaptation (LoRA) 3
LoRA Variant / Extension 5
Implementation 5
Environment / Libraries 5
Weights and Biases 6
Full Fine Tuning 6
LoRA Fine Tuning 6
Results and Analysis 7
Raw Training Results 7
Evaluation On Test Dataset / Analysis 8
Parameter Reduction / GPU Memory Efficiency 10
References 12

Whisper STT

Whisper STT (speech to text) is a transformer-based model for automatic speech recognition (ASR). It is an encoder-decoder model, also referred to as a sequence-to-sequence model. It was developed by OpenAI in 2022 and showcased considerable improvements at that time. Whisper applies features extraction in the form of converting raw wav audio to mel spectrogram which is then passed into the transformer. For this project, we have chosen to fine tune it's 39M param tiny model, which supports English only STT.

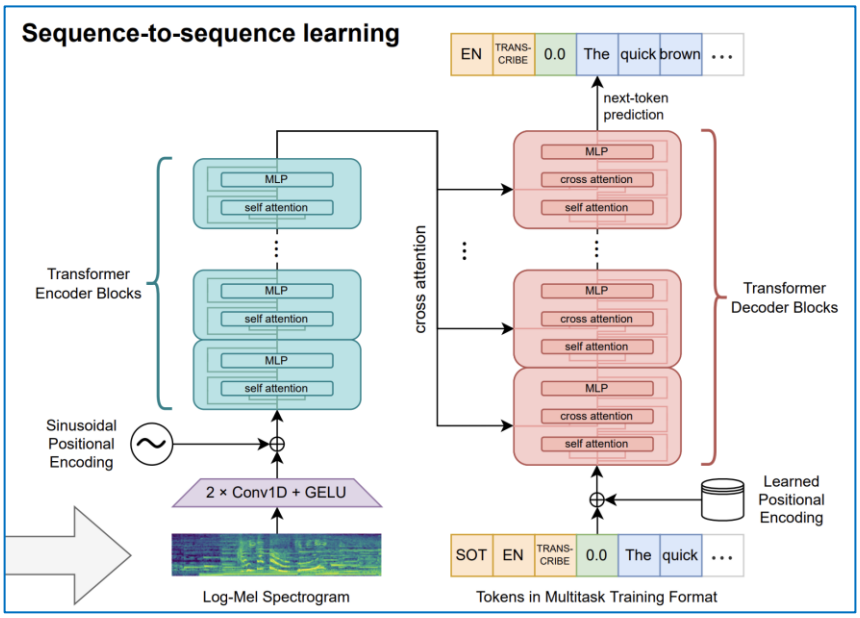

Fig 1: Whisper Transformer Architecture

Model	Layers	Width	Heads	Parameters
Tiny	4	384	6	39M
Base	6	512	8	74M
Small	12	768	12	244M
Medium	24	1024	16	769M
Large	32	1280	20	1550M

z

Fig 2: Whisper Models Sizes

Task / Dataset Selection

The dataset we have decided to train on is a set of audio recordings from Trump speeches from Kaggle (Eyas Taifour, 2019). Our goal is to improve the models WER (word error rate)

performance on Trumps voice. The dataset contained four different speeches, ranging from roughly 30 – 90 minutes long. Significant preprocessing was required to prepare the dataset for fine tuning:

1. **Audio segmentation:** Whisper STT is trained on 5 – 30 second audio segments. Training on varying lengths improves generalization.
2. **Padding:** Whisper trains on 30 second segments. Audio less than 30 seconds is padded with zeros, representing blank audio. Training on blank audio ensures our model does not suffer from "trigger happy decoding" ie predicting false positives.
3. **Segmentation strategy:** Audio is split into sentences. In cases where sentences are over 30 seconds long, we look for logical breaks like commas in the audio transcript.
4. **Data split:** train (80%), validation (10%), test (10%), done randomly

```
✅ Created 756 total segments.
Train: 604, Val: 75, Test: 77
```

Fig 3: Dataset Split

The key metric for benchmarking the performance of the model is WER. This metric calculates the ratio of words predicted by the model that are incorrect. It is also worth noting that the transcripts in our dataset are not without issues. On inspection, some discrepancies have been found between the audio and transcript.

Low Rank Adaptation (LoRA)

LoRA (Low-Rank Adaptation) (Hu et al., 2021) is a parameter-efficient fine-tuning technique that enables model adaptation with minimal computational overhead. Instead of updating all parameters of a model, LoRA introduces small low-rank matrices to capture task-specific knowledge while freezing the original model weights.

<u>During Training:</u>

We start with a pretrained weight matrix, which is frozen (not updated during training). The goal is to adapt this matrix to the new data without fully fine-tuning W, which would be computationally expensive and require storing many parameters.

Instead, we inject a low-rank update:

$h = Wx + BAx$

Effectively, instead of learning a full W, we learn BA, where A and B are much smaller than W, dramatically reducing the number of parameters. The low-rank update BAx is added to the frozen output Wx, allowing the model to adapt without modifying W.

After Training:

After training is complete, the low-rank update can be merged with the original weights:

W(merged) = W + BA

So, the updated equation becomes:

h = W(merged)x

This simplifies inference because the two matrices can be merged into one.

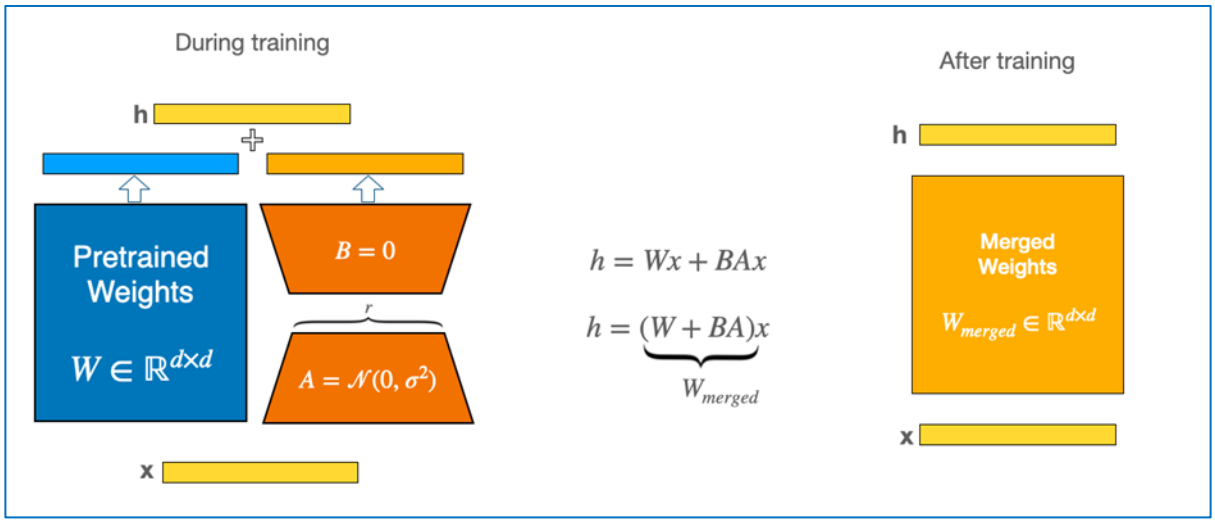

Fig 4: LoRA Diagram

Advantages of LoRA:

- Computational Efficiency: LoRA significantly reduces memory usage, making it feasible to fine-tune models on resource-constrained hardware.
- Faster Convergence: Since only a small number of parameters are modified, lora fine-tuning converges faster than full model adaptation.
- Preservation of General Knowledge: By keeping the original model weights intact, LoRA ensures that general capabilities learned during pretraining are not lost.

Limitations of LoRA:

- Hyperparameter sensitivity: LoRA fine-tuning performance is highly dependent on rank, scaling, and learning rate configuration
- Limited adaptation capacity: Only a small number of low-rank matrices are trained, which may not capture complex task-specific patterns
- Low benefit for very small models: When the model size is small, LoRA's memory / computation savings are minimal.

LoRA Variant / Extension

AdaLoRA (Zhang et al., 2023) is a promising extension on LoRA, which introduces an adaptive budget allocation mechanism for low-rank fine-tuning rather than a uniform treatment of all layers. Whereas our current implementation of LoRA statically assigns the same rank and scaling factor across chosen modules, AdaLoRA instead assesses the 'importance' of each weight matrix and dynamically distributes the avialable parameter budget (low-rank updates) accordingly. Specifically, the method parameterizes low-rank increments via singular value decomposition and then prunes singular values of less-important updates, improving parameter efficiency and performance under tight budgets. In the context of ASR, adopting AdaLoRA would allow us to allocate more adaptation capacity to those layers that most strongly govern speaker specific acoustic or linguistic idiosyncrasies, while reducing adaptation on more generic layers. This has potential to further reduce memory / training time while preserving WER compared to LoRA.

Implementation

Environment / Libraries

At the beginning of this project, we were using Google Colab for development. However, unsupported library versions became an issue, which forced us to change to a local development environment instead. This allowed us to resolve issues with library versions. We performed fine tuning on Windows (CPU) and Mac (mps) laptops. We also had access to an NVIDIA RTX A4500 GPU. Several external libraries were used to setup the model for fine tuning:

- PyTorch (Pytorch, 2023)
- PEFT (parameter efficient fine tuning)
- Hugging Face transformers

Other important aspects of implementation include:

- **Custom Data Collator –** this class pads audio to reach this threshold and
- **Whisper Processor –** the processor includes a feature extractor for resampling audio at 16k Hz and converting the raw audio to mel spectrogram. It also includes a tokenizer for converting transcriptions to token ids. Each token id is later converted to a vector embedding.
- **Adam optimizer –** the hugging face library used implements Adam by default under the hood.

Weights and Biases

To identify the optimal hyperparameter configurations for full and LoRA fine-tuning, we leveraged W&B's sweep functionality. Two separate sweeps were conducted, one for LoRA and one for full fine-tuning, each configured to minimize the validation WER. We used the Bayesian optimization method, which efficiently explores the parameter space by iteratively selecting new configurations based on the performance of previous runs. The sweep experiments were executed on an NVIDIA RTX A4500 GPU. Each combination from the sweeps was logged and visualized in W&B dashboards, allowing us to then select the hyperparameters which resulted in the lowest WER & validation loss for full and LoRA respectively. We also used W&B to log and visualize metrics from individual fine-tuning runs.

Full Fine Tuning

For full fine-tuning, all model parameters are unfrozen and updated. The training uses the Hugging Face Seq2SeqTrainer, with evaluation performed before and after training. We opted to use a learning rate of 2e-05, a batch size of 4, 8 epochs, and 50 warmup steps (values obtained from W&B sweep). By default, Seq2SeqTrainer uses AdamW (Adam with decoupled weight decay), which is well-suited for transformer models. The training time required for full tine tuning was notably longer than the training time required for LoRA fine-tuning, the amount by which it differed varied depending on the hardware setup used. The memory required for full fine-tuning compared to LoRA was approximately double.

LoRA Fine Tuning

For LoRA fine tuning, only the attention projection (q_proj, v_proj) layers of the Whisper model are adapted using low-rank matrices. This reduces the number of trainable parameters to approximately 1.5% of that of the full fine-tuning approach. We used a learning rate of 7e-05, a batch size of 4, 8 epochs, and 0 warmup steps. We set lora_alpha to 32, lora_rank to 32, and lora_dropout to 0.05 respectively. Again, these values were obtained from the results of running a sweep using W&B. The get_peft_model function wraps the base model with the LoraConfig, inserting LoRA layers that learn low-rank weight updates while the original model weights remain frozen. The HuggingFace Seq2SeqTrainer orchestrates training and evaluation, and the fine-tuned LoRA adapter weights and processor are saved after training.

Results and Analysis

Raw Training Results

The results of training Whisper Tiny on our dataset using full fine tuning and LoRA can be seen in figures 5 and 6 below.

Step	Training Loss	Validation Loss	Model Preparation Time	Wer
25	2.054000	1.631569	0.001000	0.257159
50	1.108400	1.109075	0.001000	0.237507
75	0.746900	0.852558	0.001000	0.265020
100	0.313300	0.689202	0.001000	0.225716
125	0.314900	0.685190	0.001000	0.258843
150	0.322900	0.676325	0.001000	0.266143
175	0.166200	0.686389	0.001000	0.249298
200	0.187300	0.698241	0.001000	0.254351
225	0.197900	0.686311	0.001000	0.279057
250	0.138500	0.710217	0.001000	0.255474
275	0.125000	0.717010	0.001000	0.243683
300	0.106600	0.709996	0.001000	0.256036
325	0.047700	0.740199	0.001000	0.202134
350	0.079700	0.733777	0.001000	0.218417
375	0.090500	0.736255	0.001000	0.213363
400	0.050700	0.751545	0.001000	0.266143
425	0.059600	0.753750	0.001000	0.218417
450	0.022500	0.760019	0.001000	0.257159
475	0.028900	0.767388	0.001000	0.257720
500	0.036800	0.771509	0.001000	0.255474
525	0.035300	0.774669	0.001000	0.255474
550	0.021500	0.779985	0.001000	0.259405
575	0.018200	0.783169	0.001000	0.258282
600	0.026600	0.784366	0.001000	0.259405

Fig 5: Full Fine-Tuning Results

On initial inspection of our full-fine tuning, we see a gradual decrease in validation loss. Notice that the decrease in WER jumps up and down at first. This is most likely due to a small validation dataset of only 75 samples, as well as the sensitive nature of the WER metric. If one word is incorrectly predicted, the next word is also likely to be. Note that we implement early stopping by adding a parameter *'load_best_model_at_end=True'*, saving the best performing WER models' weights. Above the weights after 325 steps are saved.

Step	Training Loss	Validation Loss	Model Preparation Time	Wer
25	2.599600	2.684235	0.001700	0.287479
50	2.426000	2.300544	0.001700	0.275126
75	1.870300	1.941783	0.001700	0.276811
100	1.503400	1.595562	0.001700	0.262774
125	1.243100	1.382528	0.001700	0.289163
150	1.132700	1.312481	0.001700	0.261089
175	0.987400	1.284402	0.001700	0.252667
200	1.060400	1.262638	0.001700	0.250421
225	1.018100	1.249090	0.001700	0.245929
250	1.054100	1.237658	0.001700	0.241999
275	1.010100	1.226668	0.001700	0.243683
300	0.932100	1.217961	0.001700	0.244245
325	0.877200	1.208615	0.001700	0.244806
350	0.987200	1.204492	0.001700	0.244806
375	0.992100	1.199229	0.001700	0.244245
400	0.966200	1.194559	0.001700	0.244806
425	0.953300	1.189062	0.001700	0.244806
450	0.851700	1.184994	0.001700	0.243683
475	0.870100	1.180865	0.001700	0.239753
500	0.951600	1.177253	0.001700	0.237507
525	0.912900	1.176445	0.001700	0.237507
550	0.932400	1.175020	0.001700	0.239191
575	0.850700	1.173729	0.001700	0.239191
600	0.929100	1.173508	0.001700	0.239191

Fig 6: LoRA Fine-Tuning Results

Figure 6 shows LoRA fine-tuning producing a steady decrease in validation loss up to 550 steps where from then on, the loss then begins to plateau. We see a corresponding decrease in WER on our validation dataset.

Evaluation On Test Dataset / Analysis

Both of the final fine-tuned models were evaluated on the test dataset. The results can be seen in figure 7 below. We report a significant reduction in loss and WER in both our models. Test loss drops from 2.67 in the base model to 0.4 in the fully fine-tuned model and 0.89 in the LoRA fine-tuned model. More importantly, we see a decrease of 6% in our key WER metric.

```
[20/20 00:05]

[FULL] After fine-tuning → Test Loss: 0.4012, WER: 0.165
[FULL] After fine-tuning → Val Loss: 0.6892, WER: 0.226

[20/20 00:06]

[LoRA] After fine-tuning → Test Loss: 0.8946, WER: 0.175
[LoRA] After fine-tuning → Val Loss: 1.1773, WER: 0.238
```

Figure 7: Full and LoRA Fine-Tuning Results

```
[20/20 00:11]

Before fine-tuning → Test Loss: 2.6792, WER: 0.230
Before fine-tuning → Val Loss: 2.8140, WER: 0.291
```

Figure 8: Base Tiny Model Results

Initially, when running the model on our test dataset, we noticed a much lower WER and loss than what was reported by our model on validation during training. This had us debugging for issues in WER implementation and code. After adding extra logging and evaluating the model on our test and validation dataset for both pre and post training, we concluded that our test dataset just so happened to be an easier dataset to transcribe. Figures 7 and 8 now also show the loss and WER of the models on the validation set for pre and post training. We notice a higher loss and WER on the validation set for the base model and both fine-tuned models, confirming our hypothesis that the validation set is just more difficult than the test, and the differences between training validation and test metrics are justified.

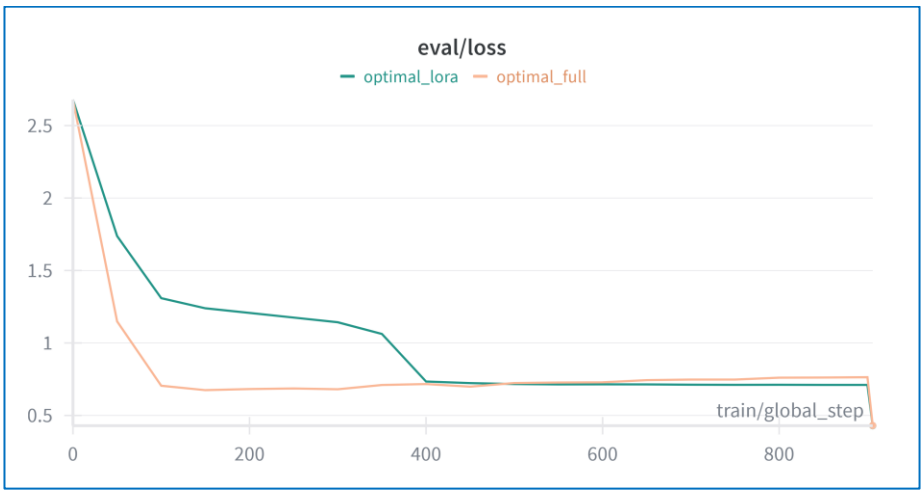

Figure 9: Validation Loss

Figure 9 above and figure 8 below depict the validation and training loss decreasing as the model converges. A key takeaway here is that on both occasions, the full fine-tuning results in the loss decreasing faster. This is a trade-off between memory efficiency and speed of convergence.

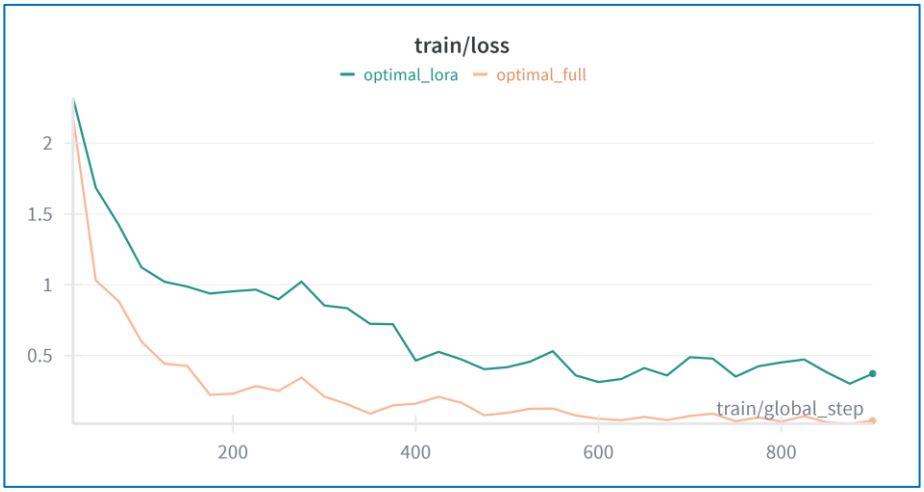

Figure 10: Training Loss

Parameter Reduction / GPU Memory Efficiency

Method	Total Number of Parameters	No. Of trainable Parameters	% trainable parameters
Full	37,760,640	37,760,640	100%
LoRA	37,760,640	589,824	~1.54%

Figure 11: Parameter efficiency

This table illustrates the difference in parameter efficiency between full fine-tuning and LoRA. Despite the significant reduction in trainable parameters, LoRA adapts the model effectively by learning low-rank updates within selected attention layers rather than modifying all parameters directly. This reduction translates to significantly lower GPU memory usage, as illustrated below.

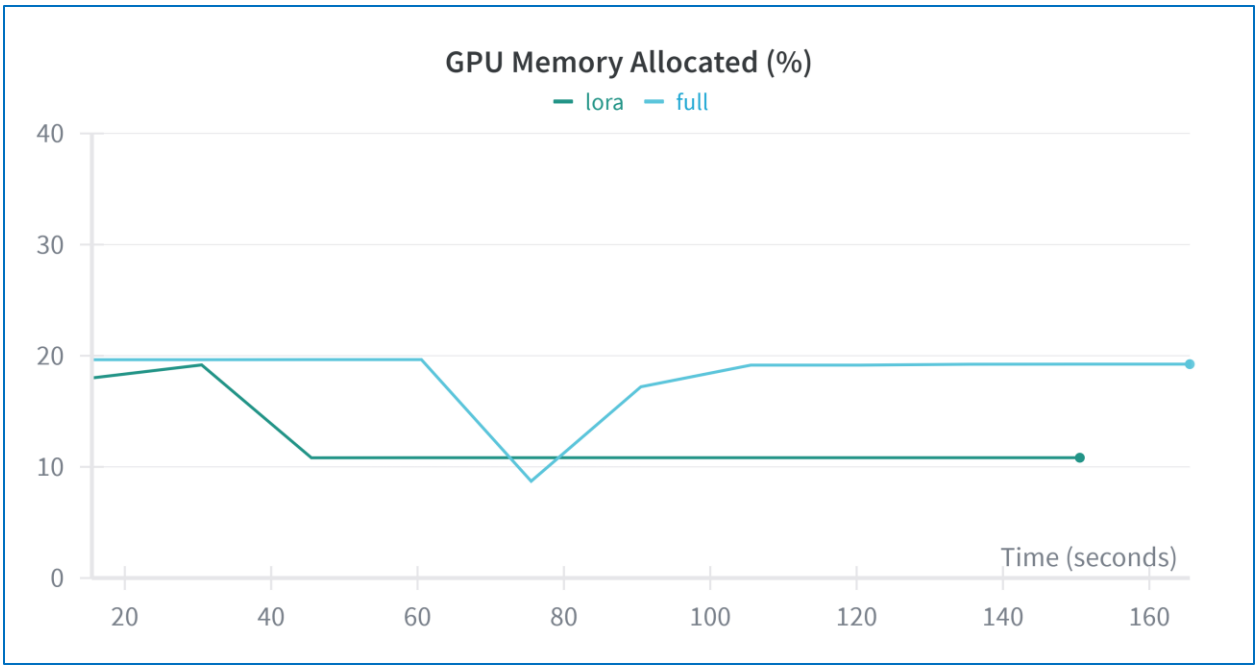

Figure 12: GPU Memory allocated

Across the course of training, LoRA fine-tuning required approximately half the GPU memory compared to full fine-tuning on an NVIDIA RTX A4500 GPU. Training also took about 10 seconds less for LoRA compared to full fine tuning. While these gains may appear small, for larger models, the impact is significant.

References

Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. and Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *arXiv:2106.09685 [cs]*. [online] Available at: https://arxiv.org/abs/2106.09685.

Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C. and Sutskever, I. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. *arXiv:2212.04356 [cs, eess]*. [online] Available at: https://arxiv.org/abs/2212.04356.

Eyas Taifour (2019). *Trump Speeches (audio and word transcription)*. [online] Kaggle.com. Available at: https://www.kaggle.com/datasets/etaifour/trump-speeches-audio-and-word-transcription

Huggingface.co. (2023). *openai/whisper-tiny · Hugging Face*. [online] Available at: https://huggingface.co/openai/whisper-tiny.

OpenAI (2022). *Whisper*. [online] GitHub. Available at: https://github.com/openai/whisper.

PyTorch (2023). *PyTorch*. [online] Pytorch.org. Available at: https://pytorch.org/.

huggingface.co. (n.d.). *PEFT*. [online] Available at: https://huggingface.co/docs/peft/en/index.

huggingface.co. (n.d.). *Transformers*. [online] Available at: https://huggingface.co/docs/transformers/en/index.

Zhang, Q., Chen, M., Bukharin, A., He, P., Cheng, Y., Chen, W. and Zhao, T. (2023). *Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning*. [online] arXiv.org. doi:https://doi.org/10.48550/arXiv.2303.10512.